

Frontend Design Prompt Kit

AstroPay-inspired prompts for Ultraviolet's student booking site, Stellar USDC checkout, auth and identity flows, and admin operations dashboard.

Reference

AstroPay's current navigation, global payment trust signals, exchange widgets, and business payment pages.

Repo Structure

```
ultraviolet-backend,  
ultraviolet-frontend,  
ultraviolet-ops.
```

Output

Copy-paste implementation prompts with explicit upgrades and no blockchain gimmicks.

Prepared on May 17, 2026. AstroPay reference pages reviewed: homepage, business page, accept-payments page, and developer docs landing page.

Read This First

Ruthless boundary: AstroPay and the earlier Stargate idea are references, not things to clone. Ultraviolet is an education operations product with backend-verified Stellar payments. Do not turn it into a token, NFT, credential, or Soroban demo before payment creation, verification, reconciliation, auth, and admin auditability work.

AstroPay is useful because it shows how a money product makes global payments feel simple: clear navigation, trust proof, exchange-rate interaction, concise feature strips, and business payment positioning. Ultraviolet should borrow the clarity, not the brand. The UI must serve students booking education consulting services and admins reconciling payment status against backend-owned truth.

Repo Structure Reference

Use the three-repo reference structure as a separation-of-concerns model, but keep Ultraviolet's real split. A contract repo is not justified for the MVP.

ultraviolet-backend

Rust backend for payment truth, auth, identity, reconciliation, admin APIs, and audit logs.

- Owns invoice/payment state.
- Verifies Stellar facts server-side.
- Protects student PII.

ultraviolet-frontend

Next.js frontend for public booking, checkout, admin dashboard, and auth flows.

- Displays backend status.
- Collects student intake.
- Never decides payment truth.

ultraviolet-ops

Deployment, environment contracts, runbooks, production gates, and incident response.

- Database and service orchestration.
- Reconciliation runbooks.
- Security and launch checklists.

Header rule: brand docs and README headers as `NUMBER / Ultraviolet` until final brand identity is locked.

AstroPay Reference Audit

Patterns Worth Reusing

- Top-level navigation that separates user types and product lines.
- Dark hero with a direct borderless-money promise.
- Exchange-rate widget near the hero as proof of utility.
- Trust proof near conversion CTAs.
- Simple action verbs for product sections.
- Business page focused on reach, cost, speed, and developer integration.

Ultraviolet Must Improve

- AstroPay is broad fintech. Ultraviolet needs education-intake specificity.
- Ratings are not enough. Show backend status, payment status, and admin auditability.
- Checkout must explain Stellar verification without exposing students to protocol noise.
- Auth and identity need first-class UI, not a login modal afterthought.
- AI output is advisory metadata only, never payment or eligibility truth.
- No frontend state may downgrade or verify payment status.

Prompt 00 - Preflight Boundary

00-A / Audit Before Building UI

Audit the Ultraviolet frontend task before writing code.

Repos:

- ultraviolet-backend owns auth, payment truth, reconciliation, applicant records, and audit logs.
- ultraviolet-frontend owns public booking UI, checkout UI, admin UI, and display-only status.
- ultraviolet-ops owns environment contracts, deployment, and runbooks.

Required output:

- Which repo owns the change.
- Which backend endpoint or mock contract the frontend depends on.
- Which PII fields are touched.
- Which payment states are display-only.
- Which auth role can access the screen.
- Which AstroPay pattern is being adapted.
- Which pattern must be rejected because it is brand-specific, consumer-only, or unsafe.

Hard rules:

- Do not use AstroPay logos, images, exact text, or protected trade dress.
- Do not create a contract repo for the MVP.
- Do not mark payments paid, verified, settled, or failed from frontend-only state.
- Do not expose applicant/admin records through public routes.

Prompt 01 - Design System

01-A / Ultraviolet Tokens

FRONTEND

DESIGN SYSTEM

Create the Ultraviolet design-token foundation for a Next.js App Router app.

Reference AstroPay's confidence and simplicity, but create an original education-fintech identity.

Required tokens:

- Background dark: #101525.
- Surface dark: #18203A.
- Surface light: #F7F9FC.
- Primary action: #5F4AE8.
- Success / verified: #008F8C.
- Pending / review: #B7791F.
- Error / failed: #C94F3D.
- Education accent: #2F6FED.
- Text primary dark: #FFFFFF.
- Text primary light: #121826.
- Muted text: #556070.
- Border: #D8E0EA.

Output:

- tailwind.config.ts theme extension.
- app/globals.css CSS variables.
- semantic aliases for payment, applicant, and auth states.
- dark mode via class strategy.
- Storybook token page that renders all colors, radii, shadows, and status colors.

Upgrade: use semantic state tokens. The payment UI must remain truthful even if the brand palette changes.

NUMBER / ULTRAVIOLET FRONTEND DESIGN PROMPT KIT

Build Ultraviolet's typography, spacing, radius, and motion primitives.

Requirements:

- Font: Inter via next/font for UI.
- Mono: JetBrains Mono for payment IDs, Stellar memos, wallet addresses, and audit IDs.
- Type scale: display 48, h1 36, h2 28, h3 22, body 16, small 13, label 11 uppercase.
- Spacing: 4px base grid with 4, 8, 12, 16, 24, 32, 48, 64, 96.
- Radius: 4px controls, 8px cards, 12px dialogs, full pills.
- Motion: 120ms controls, 180ms drawers, 240ms modals, reduce-motion support.

Output:

- app/fonts.ts.
- CSS utility layer.
- Storybook page showing typography on light and dark surfaces.

Do not scale font size with viewport width. Text must not overflow buttons, sidebar labels, status badges, or table cells.

Prompt 02 - Header And Navigation

02-A / Public Header Branded As NUMBER / Ultraviolet

HEADER**NUMBER**

Build the Ultraviolet public header inspired by AstroPay's clear product navigation.

Brand requirement:

- Top-left lockup reads: NUMBER / Ultraviolet.
- Do not use AstroPay colors, logos, product labels, or copy verbatim.

Desktop:

- Left: NUMBER / Ultraviolet logo lockup.
- Center nav: Services, How it works, Payments, Institutions, Resources.
- Payments menu: Stellar USDC checkout, Payment status, Refund policy, Security.
- Resources menu: Student guide, Counselor guide, FAQs, Contact.
- Right: Student sign in, Admin sign in, Book consultation.
- On scroll: accessible backdrop-blur header with 1px border.

Mobile:

- Hamburger opens full-height drawer.
- Drawer uses accordion groups.
- Bottom sticky CTA: Book consultation.
- Escape closes drawer, focus returns to hamburger, and body scroll is locked.

Upgrade: split student and admin entry points clearly. A public education site with hidden admin access becomes messy fast.

NUMBER / ULTRAVIOLET FRONTEND DESIGN PROMPT KIT

Create the authenticated Ultraviolet admin header.

Layout:

- Left: page title and environment badge, Local / Testnet / Production.
- Center: Cmd+K search for applicants, payment requests, notes, and tasks.
- Right: notifications, support menu, admin role badge, user avatar.

Rules:

- Testnet and Production must be visually impossible to confuse.
- Avatar menu includes Profile, Security, Team, Audit log, Sign out.
- Display session status from backend-owned /me endpoint only.
- Do not store admin credentials or tokens in localStorage.

Prompt 03 - Public Site

03-A / Hero And Trust Strip

Build the Ultraviolet homepage hero.

Reference:

- AstroPay leads with a clear borderless-money promise, app CTAs, ratings, and an exchange widget.

Ultraviolet version:

- H1: "Book education support. Pay globally. Stay in control."
- Supporting copy explains education consulting, secure intake, Stellar USDC payment requests, and admin-reviewed progress.
- Primary CTA: Book consultation.
- Secondary CTA: Check payment status.
- Right side: real product preview showing service booking, payment request, and admin status timeline.
- Below hero: trust strip with "PII off-chain", "Backend-verified payments", "Admin-reviewed intake", "Stellar USDC supported".

Do not invent fake production metrics. If metrics are examples, label them as demo values.

Upgrade: AstroPay sells broad money movement. Ultraviolet must sell trust around student data and payment verification.

NUMBER / ULTRAVIOLET FRONTEND DESIGN PROMPT KIT

Create the public service sections.

Sections:

- Choose a service: consultation, application review, institution matching, visa-prep support.
- Submit intake: goals, destination, timeline, documents status, contact details.
- Pay securely: Stellar USDC request with backend verification.
- Track progress: admin-reviewed tasks and next steps.

Design:

- Four concise feature panels, not a decorative card wall.
- Use real workflow visuals, not abstract blobs.
- Include an institution discovery section with filters by country, program type, and budget range.
- Include FAQs for payment status, refunds, privacy, and data retention.

Accessibility:

- All CTAs have visible focus.
- Motion respects prefers-reduced-motion.
- Payment state is not communicated by color alone.

Prompt 04 - Auth And Identity

04-A / Student And Admin Auth

AUTH

IDENTITY

FRONTEND

Build Ultraviolet auth screens.

Routes:

- /login
- /register
- /forgot-password
- /reset-password
- /verify-email
- /admin/login
- /admin/mfa

Design:

- Use NUMBER / Ultraviolet header on auth pages.
- Forms must be compact, trustworthy, and password-manager friendly.
- Student and admin entry points must be visually distinct.
- Admin login must support MFA.

Security UX:

- Never store tokens in localStorage.
- Use secure HTTP-only cookie session assumed from ultraviolet-backend.
- Show generic invalid-login errors.
- Rate-limit messaging must not reveal whether an email exists.
- MFA screen supports authenticator code and recovery code.

Output:

- Typed form components.
- zod schemas.
- Loading, disabled, success, and error states.
- Tests for keyboard submission and error rendering.

NUMBER / ULTRAVIOLET FRONTEND DESIGN PROMPT KIT

Design the student identity and consent flow.

Routes:

- /intake/profile
- /intake/education
- /intake/goals
- /intake/documents
- /intake/review

Steps:

1. Contact profile: name, email, phone, country.
2. Education background: current level, target program, grades summary.
3. Goals: destination countries, budget range, timeline, support type.
4. Documents status: passport, transcript, language test, visa records as status only unless upload policy exists.
5. Consent and review: explain data usage before submission.

Rules:

- Each step autosaves through backend endpoints.
- Sensitive fields explain why they are needed.
- Mask sensitive values after save.
- Do not send sensitive documents to AI APIs without explicit consent and policy.
- AI output is advisory metadata only.

NUMBER / ULTRAVIOLET FRONTEND DESIGN PROMPT KIT

Build the Team and Access page for admins.

Features:

- Invite teammate modal.
- Roles: Owner, Admin, Counselor, Finance, Viewer.
- Permission matrix with plain-language descriptions.
- Pending invites table.
- Active sessions table for the current user.
- Remove user confirmation dialog requiring typed organization name.

Frontend rules:

- Hide unavailable actions but also handle backend 403 responses gracefully.
- Do not rely on frontend checks for authorization.
- Show audit log links for security-sensitive actions.
- Include empty states and skeletons.

Required tests:

- Unauthorized role cannot see destructive actions.
- Backend 403 shows a clean error state.
- Invite form validates missing email and invalid role.

Prompt 05 - Booking And Checkout

05-A / Appointment Booking Flow

Build the student appointment booking flow.

Steps:

1. Select service.
2. Select counselor or "first available".
3. Pick date and time.
4. Fill intake summary.
5. Review service fee.
6. Create payment request.

Requirements:

- Mobile-first.
- Save draft after each step.
- Show timezone clearly.
- Validate required fields with zod.
- Use backend-created payment request; do not calculate final payment truth in the browser.
- Show privacy note before collecting PII.

Upgrade over AstroPay:

- Pair payment UX with education workflow context so students know what happens after paying.

NUMBER / ULTRAVIOLET FRONTEND DESIGN PROMPT KIT

Build hosted checkout at `/pay/[publicId]`.

Layout:

- Mobile-first max width 480px.
- Ultraviolet logo, service summary, payment amount, and payment memo.
- "Secured by Ultraviolet" with a concise explanation: PII stays off-chain; payment is verified by backend.
- Freightier wallet action if available.
- Manual payment instructions with destination, asset, amount, and memo.
- Copy buttons for destination and memo.
- QR code if backend provides payment URI.

States:

- Loading request.
- Expired request.
- Waiting for payment.
- Submitted by wallet.
- Verifying on Stellar.
- Verified by backend.
- Failed verification.
- Backend unavailable.

Hard rules:

- The browser submits transaction evidence, then waits for backend verification.
- Do not show success just because a wallet submitted.
- Unknown transaction hash must not stay pending forever without retry or expiration copy.

NUMBER / ULTRAVIOLET FRONTEND DESIGN PROMPT KIT

Create checkout result screens.

Success:

- "Payment verified" headline.
- Amount, asset, payment ID, and transaction hash.
- Receipt download if backend supports it.
- Next step: "Your counselor will review your intake."

Pending:

- Verification checklist: transaction exists, successful, destination, amount, asset, memo, hash uniqueness.
- Poll backend status every 3 seconds with backoff.

Failure:

- Human-readable reason such as wrong memo, wrong amount, wrong asset, duplicate hash, not found, or network unavailable.
- Try again.
- Contact support.
- Preserve checkout state.

Accessibility:

- Status changes announced via aria-live.
- No decorative success animation if reduce-motion is enabled.

Prompt 06 - Admin Dashboard

06-A / Dashboard Shell

Create the Ultraviolet admin dashboard shell.

Layout:

- Left sidebar 248px expanded, 68px collapsed.
- Sidebar sections: Overview, Applicants, Appointments, Payments, Tasks, Institutions, AI Notes, Audit Log, Settings.
- Sidebar header: organization switcher if multi-office support exists.
- Sidebar footer: environment, role, support status.
- Mobile: bottom tab bar for Overview, Applicants, Payments, Tasks, Settings.

Visual:

- Quiet operational UI. No oversized marketing hero inside the dashboard.
- 8px card radius.
- Dense but readable tables.
- Status colors map to semantic tokens.

Data:

- Use backend /me and /admin/context to hydrate app shell.
- Show skeletons until data loads.
- Production and local/testnet must be visually distinct.

NUMBER / ULTRAVIOLET FRONTEND DESIGN PROMPT KIT

Build the admin Overview page.

KPI row:

- New applicants.
- Booked consultations.
- Payment requests created.
- Verified payments.
- Payments needing review.

Main content:

- Applicant funnel chart: intake started, submitted, payment requested, paid, counselor assigned.
- Recent applicants table.
- Recent payments table.
- Alerts panel for failed verification, duplicate hashes, expired requests, missing intake fields.

Rules:

- Use sample data only behind clearly named demo mode.
- All money is rendered from minor units or precise decimal strings.
- A payment can only display "verified" if backend status says `verified_on_stellar`.

Upgrade: AstroPay-style metrics are money-centric. Ultraviolet needs a combined operations view: applicants plus payment risk.

NUMBER / ULTRAVIOLET FRONTEND DESIGN PROMPT KIT

Build an applicant detail drawer.

Trigger:

- Admin clicks an applicant row.

Content:

- Applicant summary with PII masked where appropriate.
- Intake completeness checklist.
- Service request.
- Payment request status.
- Payment verification timeline.
- Counselor notes.
- AI advisory summary with "human review required" label.
- Audit events for viewed/changed applicant state.

Actions:

- Assign counselor.
- Request missing information.
- Create payment request.
- Mark internal task complete.

Hard rules:

- Do not expose applicant detail through public routes.
- Do not leak full PII in logs or UI copy intended for screenshots.

Prompt 07 - Payment Operations

07-A / Payment Requests Manager

Build the Payment Requests page.

List:

- Search, filters, column visibility, pagination.
- Columns: Payment ID, Applicant, Service, Amount, Asset, Memo, Status, Expires, Transaction Hash, Created.

Create modal:

1. Select applicant and service.
2. Confirm amount and asset.
3. Generate backend-owned memo.
4. Review destination account and expiry.
5. Share payment link.

Validation:

- Backend generates payment ID and memo.
- Frontend cannot edit memo after creation.
- URL fields require HTTPS except localhost in development.

Statuses:

- created
- awaiting_payment
- submitted_for_verification
- verified_on_stellar
- failed_verification
- expired
- cancelled

Do not allow a verified payment to be downgraded from UI actions.

NUMBER / ULTRAVIOLET FRONTEND DESIGN PROMPT KIT

Build the payment verification detail drawer.

Content:

- Expected destination.
- Expected amount.
- Expected asset.
- Expected memo.
- Submitted transaction hash.
- Backend verification result.
- Failure reason if any.
- Reconciliation attempts and timestamps.
- Duplicate hash warning if applicable.

Checklist display:

- Transaction exists on configured Stellar network.
- Transaction successful.
- Hash unused.
- Destination matches.
- Amount matches.
- Asset matches.
- Memo matches.
- Payment request ownership matches.
- Fee-bump, muxed account, path payment, and multi-operation policy handled.

Only backend data can check an item. The frontend must not infer checklist success locally.

Prompt 08 - AI Advisory Workflow

08-A / Intake Quality Assistant

Build AI advisory UI for admin intake review.

Allowed AI outputs:

- Missing intake fields.
- Unclear student goals.
- Suggested follow-up questions.
- Duplicate or suspicious lead flag.
- Service type classification.
- Admin response draft for human review.

UI:

- AI Notes panel inside applicant detail.
- Advisory badge, never truth badge.
- "Accept suggestion", "Dismiss", and "Edit before sending" actions.
- Show model/source timestamp if backend provides it.

Forbidden:

- Do not use AI to verify payment.
- Do not use AI to make final visa, legal, financial, or admission eligibility decisions.
- Do not send sensitive documents to third-party AI without explicit consent and data policy.

Prompt 09 - Shared Components

09-A / Status Badge

Create StatusBadge.

States:

- intake_started
- intake_submitted
- payment_requested
- awaiting_payment
- submitted_for_verification
- verified_on_stellar
- failed_verification
- expired
- assigned
- review_required

Props:

- status
- size: sm | md | lg
- showIcon
- pulse

Rules:

- Use lucide-react icons.
- Include text labels for accessibility.
- Color is never the only state indicator.
- Storybook covers all states in light and dark mode.

NUMBER / ULTRAVIOLET FRONTEND DESIGN PROMPT KIT

Create MoneyDisplay.

Props:

- amountAtomic or decimalString
- assetCode
- issuerLabel
- locale
- showCode
- size
- intent: default | positive | negative | muted

Rules:

- Do not accept JavaScript floating point numbers for money.
- Stellar assets preserve 7-decimal precision where relevant.
- Fiat display uses Intl.NumberFormat only for presentation.
- Monospace option for reconciliation rows.

09-C / Data Table

Create DataTable compound component.

Features:

- Sort.
- Filter.
- Pagination.
- Column visibility.
- Bulk select.
- Empty states.
- Skeleton states.
- Row action menu.

Use it for:

- Applicants.
- Appointments.
- Payment requests.
- Audit logs.
- Admin users.

NUMBER / ULTRAVIOLET FRONTEND DESIGN PROMPT KIT

Create EmptyState and Toast system.

EmptyState:

- icon
- title
- description
- action
- variant: page | card | table

Toasts:

- success, info, warning, error.
- optional action button.
- undo support for reversible actions.
- aria-live announcements.

No empty state should explain the whole product. It should tell the user the next action.

Prompt 10 - Quality Gate

10-A / Frontend Acceptance Tests

Create frontend acceptance tests for ultraviolet-frontend.

Required checks:

- Public header desktop and mobile navigation.
- Student auth form validation.
- Admin login failure message does not leak account existence.
- Admin dashboard requires authenticated session.
- Testnet and Production are visually distinct.
- Booking form validates required fields.
- Hosted checkout never shows success before backend verified status.
- Payment verification drawer displays all backend checks.
- AI notes are clearly advisory.
- Keyboard navigation works for drawer, modal, and menu.

Run:

- npm run lint
- npm run typecheck
- npm run test
- npm run build

If a command does not exist, add it or document why it is not available.

NUMBER / ULTRAVIOLET FRONTEND DESIGN PROMPT KIT

Review the Ultraviolet frontend before shipping.

Reject the work if:

- It visually copies AstroPay too closely.
- It uses fake metrics without a demo label.
- It lets the frontend decide payment truth.
- It hides auth, identity, or admin access edge cases.
- It cannot distinguish local/testnet/production.
- Applicant tables break on mobile.
- Text overflows buttons or badges.
- It uses decorative blobs instead of useful product visuals.
- It lacks loading, empty, error, and unauthorized states.
- It introduces a contract feature without a backend-owned payment reason.

Brutal truth: a polished education-fintech frontend that lies about payment state is worse than a plain one. Trust is a system property, not a design style.

Implementation Order

1. Put this prompt kit in `ultraviolet-frontend/docs/prompts/`.
2. Add README headers using `NUMBER / Ultraviolet`.
3. Implement tokens, header, auth, and app shell before marketing polish.
4. Build booking and checkout flows around backend-owned payment requests.
5. Add admin dashboard only after applicant and payment API contracts are clear.
6. Add tests for auth, booking validation, checkout status, and payment verification display.

Sources reviewed: <https://www.astropay.com>, <https://www.astropay.com/business>, <https://www.astropay.com/business/accept-astropay>, and <https://developers-wallet.astropay.com>. This document adapts product patterns only; it does not grant permission to reuse AstroPay protected assets.